JOB PORTAL

Full-Stack Web Application

Project Documentation

Spring Boot + React + MySQL

Technologies: Java 25, Spring Boot 4.0.4, React 19, MySQL 8.0

Architecture: RESTful API + Single Page Application

Database: Relational (14 tables)

Table of Contents

1. Introduction
2. System Architecture
3. Modules
4. Entity Relationship Diagram (ERD)
5. Use Case Diagram
6. Sequence Diagrams
7. Class Diagram
8. Database Schema
9. API Documentation
10. Technology Stack
11. Security Features
12. Future Enhancements

1. Introduction

The Job Portal is a full-stack web application designed to bridge the gap between job seekers and employers. It provides a comprehensive platform for posting job listings, submitting applications, tracking application status, communicating in real-time, managing resumes with ATS scoring, conducting skill assessments through quizzes, and administering the entire recruitment lifecycle.

The system is built using **Spring Boot 4.0.4** for the backend REST API and **React 19** with Vite for the frontend single-page application. It supports three distinct user roles with role-based access control: **Job Seeker**, **Employer**, and **Administrator**.

Key capabilities include real-time messaging via WebSocket, automated email notifications, JWT-based stateless authentication, and comprehensive analytics dashboards for both employers and administrators.

2. System Architecture

The application follows a layered architecture with clear separation of concerns across four tiers:

```
┌──────────────────────────────────────────────────────────────┐
│                    CLIENT LAYER (React SPA)                   │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐     │
│  │  React   │  │ Tailwind │  │ Zustand  │  │  Axios   │     │
│  │  v19     │  │   CSS    │  │  State   │  │  HTTP    │     │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘     │
└──────────────────────────────┬───────────────────────────────┘
                               │ HTTP REST + WebSocket (STOMP)
┌──────────────────────────────▼───────────────────────────────┐
│                   API LAYER (Spring Boot)                    │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐   │
│  │ JWT Filter   │  │ CORS Config  │  │ WebSocket Handler│   │
│  │ (Security)   │  │              │  │ (Real-time)      │   │
│  └──────────────┘  └──────────────┘  └──────────────────┘   │
│  ┌──────────────────────────────────────────────────────┐    │
│  │              REST Controllers (12)                   │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────┬───────────────────────────────┘
                               │
┌──────────────────────────────▼───────────────────────────────┐
│                    SERVICE LAYER                             │
│  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐     │
│  │ User   │ │ Job    │ │ App    │ │Message │ │Resume  │     │
│  │Service │ │Service │ │Service │ │Service │ │Service │     │
│  └────────┘ └────────┘ └────────┘ └────────┘ └────────┘     │
│  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐     │
│  │ Quiz   │ │Admin   │ │Email   │ │Notifica│ │Company │     │
│  │Service │ │Service │ │Service │ │Service │ │Service │     │
│  └────────┘ └────────┘ └────────┘ └────────┘ └────────┘     │
└──────────────────────────────┬───────────────────────────────┘
                               │ JPA / Hibernate ORM
┌──────────────────────────────▼───────────────────────────────┐
│                  DATA ACCESS LAYER                           │
│  ┌──────────────────────────────────────────────────────┐    │
│  │        Spring Data JPA Repositories (12)             │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────┬───────────────────────────────┘
                               │ JDBC
┌──────────────────────────────▼───────────────────────────────┐
│                      DATABASE                                │
│  ┌──────────────────────────────────────────────────────┐    │
│  │             MySQL 8.0 — job_portal                   │    │
│  │             14 Tables, 14 Foreign Keys               │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

3. Modules

1. Authentication & Authorization

Registration, login, JWT tokens, role-based access control (JOBSEEKER, EMPLOYER, ADMIN)

2. Job Management

Create, edit, delete, search, and filter job listings by title, location, type, salary, and experience

3. Application Management

Apply to jobs, track status through 8-stage pipeline, employer review with ratings and feedback

4. Resume Management

Upload multiple resumes, ATS scoring, keyword matching with jobs, resume organization

5. Messaging & Communication

Real-time chat between job seekers and employers, unread tracking, conversation history

6. Quiz & Assessment

Employers create skill quizzes, candidates take timed assessments, automated scoring

7. Admin Management

User management (view/edit roles/delete), job management, platform-wide statistics

8. Company Profile

Company branding with logo, banner, description, industry, size, and social links

9. Notification System

Real-time in-app notifications via WebSocket, click-to-navigate, mark as read

10. Job Alerts

User-configurable alert preferences, automatic job matching, email + in-app notifications

11. Saved Jobs

Bookmark and save jobs for later viewing and comparison

12. Analytics & Dashboard

Per-job analytics, employer aggregate metrics, application funnel tracking

4. Entity Relationship Diagram (ERD)

The database consists of 14 entities with the following relationships:

```
                    ┌──────────────┐
                    │    USER      │
                    │──────────────│
                    │ id (PK)      │
                    │ email (UK)   │
                    │ password     │
                    │ first_name   │
                    │ last_name    │
                    │ phone        │
                    │ role (ENUM)  │
                    └──────────────┘
                          │
      ┌───────────────────┼────────────────────┬────────────────────┐
      │                   │                    │                    │
  ┌───▼────┐          ┌───▼────┐          ┌───▼────┐          ┌───▼────┐
  │  JOB   │          │MESSAGE │          │ RESUME │          │NOTIFIC.│
  │────────│          │────────│          │────────│          │────────│
  │id (PK) │          │id (PK) │          │id (PK) │          │id (PK) │
  │employer│          │sender_id│         │user_id │          │user_id │
  │  id(FK)│          │receiver│          │name    │          │title   │
  │title   │          │  id(FK)│          │file_name│         │message │
  │location│          │content │          │file_path│         │type    │
  │job_type│          │is_read │          │created_│          │reference│
  │status  │          │sent_at │          │  at    │          │_id     │
  └────────┘          └────────┘          └────────┘          └────────┘
      │                                       │
  ┌───▼──────────────┐                  ┌───▼──────────────┐
  │JOB_APPLICATION   │                  │RESUME_ANALYSIS   │
  │──────────────────│                  │──────────────────│
  │id (PK)           │                  │id (PK)           │
  │job_id (FK)       │                  │user_id (FK)      │
  │jobseeker_id(FK)  │                  │resume_id (FK)    │
  │resume_id (FK)    │                  │job_id (FK)       │
  │cover_letter      │                  │score             │
  │status (ENUM)     │                  │feedback          │
  │rating            │                  │extracted_skills  │
  │feedback          │                  │matched_keywords  │
  │applied_at        │                  │missing_keywords  │
  └──────────────────┘                  └──────────────────┘
          │
      ┌───▼────┐
      │QUIZ_   │
      │RESULT  │
      │────────│
      │id (PK) │
      │applicat.│
```

```
        │  _id(FK)│
        │score    │
        │passed   │
        │time_taken│
        └─────────┘


┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│    QUIZ      │──────────▶│  QUESTION    │──────────▶│   OPTION     │
│──────────────│  1:N     │──────────────│  1:N     │──────────────│
│id (PK)       │          │id (PK)       │          │id (PK)       │
│job_id (FK)   │          │quiz_id (FK)  │          │question_id   │
│title         │          │question_text│          │  (FK)        │
│passing_score│          │score         │          │option_text   │
│time_limit    │          └──────────────┘          │is_correct    │
└──────────────┘                                     └──────────────┘


┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│  COMPANY_    │      │  SAVED_      │      │  JOB_ALERT_ │
│  PROFILE     │      │   JOB        │      │ PREFERENCE   │
│──────────────│      │──────────────│      │──────────────│
│id (PK)       │      │id (PK)       │      │id (PK)       │
│user_id (FK)  │      │user_id (FK)  │      │user_id (FK)  │
│company_name  │      │job_id (FK)   │      │keywords      │
│industry      │      │saved_at      │      │location      │
│description   │      └──────────────┘      │job_type      │
│logo_url      │                            │is_active     │
│banner_url    │                            │email_enabled│
│website       │                            │in_app_enabled│
└──────────────┘                            └──────────────┘
```

5. Use Case Diagram

Job Seeker Use Cases

- Register and login with email/password
- Search and filter jobs (title, location, type, salary, experience)
- View detailed job descriptions and company information
- Apply to jobs with cover letter and resume selection
- Track application status through 8-stage pipeline
- Take skill assessment quizzes for job screening
- Upload and manage multiple resumes with ATS scoring
- Save/bookmark jobs for later viewing
- Chat with employers in real-time
- Configure job alert preferences
- View and manage real-time notifications

Employer Use Cases

- Register and login with email/password
- Post new job listings with detailed descriptions
- Edit and manage existing job postings
- View and filter applications per job
- Update application status with ratings and feedback
- Create skill assessment quizzes for candidate screening
- Chat with candidates in real-time
- Manage company profile (logo, banner, description)
- View job analytics (views, applications, status funnel)

Admin Use Cases

- View all registered users on the platform
- Change user roles (JOBSEEKER ↔ EMPLOYER ↔ ADMIN)
- Delete users and their associated data
- View all job listings across the platform
- Delete jobs and associated applications

- View platform-wide statistics (users, jobs, applications)

6. Sequence Diagrams

6.1 User Registration Flow

```
User            Frontend        AuthController    UserService    PasswordEncoder    Database
 |                |                  |                 |                |                |
 | Enter details  |                  |                 |                |                |
 |──────────────▶|                  |                 |                |                |
 |                | POST /register   |                 |                |                |
 |                |────────────────▶|                 |                |                |
 |                |                  | register(data)  |                |                |
 |                |                  |───────────────▶|                |                |
 |                |                  |                | check email    |                |
 |                |                  |                |──────────────────────────────▶|
 |                |                  |                |◀──────────────────────────────|
 |                |                  |                | encode(password)|               |
 |                |                  |                |──────────────▶|                |
 |                |                  |                |◀──────────────|                |
 |                |                  |                | INSERT user    |                |
 |                |                  |                |──────────────────────────────▶|
 |                |                  |                |◀──────────────────────────────|
 |                |                  |                | generateToken() |               |
 |                |                  |◀──────────────|                |                |
 |                |◀────────────────|                |                |                |
 |◀──────────────|                  |                |                |                |
 | Redirect to    |                  |                |                |                |
 | Dashboard      |                  |                |                |                |
```

6.2 Job Application Flow

```
Job Seeker      Frontend        JobController    AppService    NotificationService    Database
 |                |                  |                |                |                |
 | Click "Apply"  |                  |                |                |                |
 |──────────────▶|                  |                |                |                |
 |                | GET /jobs/{id}   |                |                |                |
 |                |────────────────▶|                |                |                |
 |                |◀────────────────|                |                |                |
 | Fill form      |                  |                |                |                |
 |──────────────▶|                  |                |                |                |
 |                | POST /apply      |                |                |                |
 |                |──────────────────────────────▶|                |
 |                |                  |                | check duplicate |               |
 |                |                  |                |──────────────────────────────▶|
 |                |                  |                |◀──────────────────────────────|
 |                |                  |                | INSERT application|            |
 |                |                  |                |──────────────────────────────▶|
 |                |                  |                |◀──────────────────────────────|
```

```
  |                |                  |                  | notify employer  |                |
  |                |                  |                  |──────────────────▶|                |
  |                |                  |                  |                  | INSERT notif   |
  |                |                  |                  |                  |──────────────▶|
  |                |                  |                  |                  | WS push        |
  |                |                  |                  |◀─────────────────|                |
  |                |◀─────────────────|                  |                  |                |
  | Success!       |                  |                  |                  |                |
```

6.3 Application Status Update Flow

```
Employer        Frontend        AppController    AppService    NotificationService   Database
  |                |                  |                  |                  |                |
  | Select status  |                  |                  |                  |                |
  |────────────────▶|                  |                  |                  |                |
  |                | PUT /status      |                  |                  |                |
  |                |──────────────────▶|                  |                  |                |
  |                |                  | updateStatus()   |                  |                |
  |                |                  |──────────────────▶|                  |                |
  |                |                  |                  | UPDATE status    |                |
  |                |                  |                  |──────────────────────────────────▶|
  |                |                  |                  |◀──────────────────────────────────|
  |                |                  |                  | notify jobseeker |                |
  |                |                  |                  |──────────────────▶|                |
  |                |                  |                  |                  | INSERT notif   |
  |                |                  |                  |                  |──────────────▶|
  |                |                  |                  |                  | WS push        |
  |                |                  |                  |◀─────────────────|                |
  |                |◀─────────────────|                  |                  |                |
  | Updated!       |                  |                  |                  |                |
```

6.4 Real-Time Messaging Flow

```
Sender          WebSocket       MessageController  MessageService  Database    Receiver
  |                |                  |                  |                |            |
  | Connect STOMP  |                  |                  |                |            |
  |────────────────▶|                  |                  |                |            |
  | Subscribe      |                  |                  |                |            |
  |────────────────▶|                  |                  |                |            |
  |                |                  |                  |                |            |
  | Type & Send    |                  |                  |                |            |
  |────────────────▶|                  |                  |                |            |
  |                | SEND /chat.send  |                  |                |            |
  |                |──────────────────▶|                  |                |            |
  |                |                  | send()           |                |            |
  |                |                  |──────────────────▶|                |            |
  |                |                  |                  | INSERT message |            |
  |                |                  |                  |───────────────▶|            |
  |                |                  |                  |◀───────────────|            |
  |                |                  |                  | notify receiver|            |
```

```
    |                |                  |                  | (notification) |            |
    |                |                  |                  |                |            |
    |                |◀────────────────|                  |                |            |
    |                |◀──────────────────────────────────────────────────────────────────|
    |                | Push to receiver |                  |                |            |
    |                |──────────────────────────────────────────────────────────────────▶|
    |                |                  |                  |                | Display    |
```

7. Class Diagram

```
┌──────────────────────────┐
│         User             │
│──────────────────────────│
│ +id: Long                │
│ +email: String           │
│ +password: String        │
│ +firstName: String       │
│ +lastName: String        │
│ +phone: String           │
│ +role: UserRole (ENUM)   │
│ +profileImageUrl: String│
│ +createdAt: LocalDateTime│
│──────────────────────────│
│ +register()              │
│ +authenticate()          │
│ +updateProfile()         │
│ +changePassword()        │
└──────────────────────────┘
           │
           │
    ┌──────┼──────────────┼──────────────┼──────────────┐
    │      │              │              │              │
┌───▼───┐ ┌▼──────┐   ┌───▼───┐   ┌───▼───┐   ┌───▼───┐
│ Job   │ │Message│   │ Resume  │   │Notifica-│   │Company  │
│       │ │       │   │         │   │  tion   │   │ Profile │
│       │ │       │   │         │   │         │   │         │
│1    *│ │*    * │   │1      * │   │1      * │   │1      1 │
└───────┘ └───────┘   └─────────┘   └─────────┘   └─────────┘
    │                     │
┌───▼──────────────┐  ┌───▼──────────────┐
│  JobApplication  │  │ ResumeAnalysis   │
│──────────────────│  │──────────────────│
│ +id: Long        │  │ +id: Long        │
│ +job_id: FK      │  │ +user_id: FK     │
│ +jobseeker_id:FK│  │ +resume_id: FK   │
│ +resume_id: FK   │  │ +job_id: FK      │
│ +coverLetter     │  │ +score: Integer  │
│ +status: ENUM    │  │ +feedback: Text  │
│ +rating: Integer│  │ +skills: Text    │
│ +feedback: Text  │  │ +keywords: Text  │
│ +appliedAt       │  │ +analyzedAt      │
└──────────────────┘  └──────────────────┘
         │
    ┌───▼───┐
    │QuizResult│
    │──────────│
    │+id: Long│
    │+app_id: │
    │   FK    │
    │+score   │
```

```
    │+passed  │
    │+time    │
    └─────────┘


┌────────────┐     1:N     ┌────────────┐     1:N     ┌────────────┐
│   Quiz     │──────────▶│  Question  │──────────▶│   Option   │
│────────────│           │────────────│           │────────────│
│+id: Long   │           │+id: Long   │           │+id: Long   │
│+job_id: FK │           │+quiz_id:FK │           │+question_id│
│+title      │           │+text: Text │           │+text: Text │
│+passScore  │           │+score: Int │           │+isCorrect  │
│+timeLimit  │           └────────────┘           └────────────┘
└────────────┘
```

8. Database Schema

Table	Primary Key	Foreign Keys	Description
users	id	—	User accounts with roles, credentials, profile info
jobs	id	employer_id → users(id)	Job listings posted by employers
job_applications	id	job_id → jobs(id) jobseeker_id → users(id) resume_id → resumes(id)	Applications submitted by job seekers
messages	id	sender_id → users(id) receiver_id → users(id)	Real-time chat messages between users
resumes	id	user_id → users(id)	Uploaded resume PDFs with metadata
notifications	id	user_id → users(id)	In-app notifications with reference data
quizzes	id	job_id → jobs(id)	Skill assessment quizzes per job
questions	id	quiz_id → quizzes(id)	Questions belonging to quizzes
options	id	question_id → questions(id)	Multiple-choice options per question
company_profiles	id	user_id → users(id)	Employer company branding info
saved_jobs	id	user_id → users(id) job_id → jobs(id)	Bookmarked jobs by users
job_alert_preferences	id	user_id → users(id)	User job alert configuration
resume_analyses	id	user_id → users(id) resume_id → resumes(id) job_id → jobs(id)	ATS scoring and keyword matching results

quiz_results	id	application_id → job_applications(id)	Candidate quiz scores and pass/fail status

9. API Documentation

9.1 Authentication

Method	Endpoint	Auth	Description
POST	`/api/auth/register`	Public	Register new user (JOBSEEKER/EMPLOYER)
POST	`/api/auth/login`	Public	Login with email and password
PUT	`/api/users/{id}/password`	User	Change password

9.2 Jobs

Method	Endpoint	Auth	Description
GET	`/api/jobs`	Public	Search & filter jobs (paginated)
GET	`/api/jobs/{id}`	Public	Get job details with employer info
POST	`/api/jobs`	Employer	Create new job listing
PUT	`/api/jobs/{id}`	Employer	Update job listing
DELETE	`/api/jobs/{id}`	Employer/Admin	Delete job listing
GET	`/api/jobs/employer/{id}`	Public	Get all jobs by employer
GET	`/api/jobs/{id}/analytics`	Employer	Job analytics (views, applications)

9.3 Applications

Method	Endpoint	Auth	Description
POST	`/api/applications`	Job Seeker	Apply to a job
GET	`/api/applications/seeker/{id}`	Job Seeker	Get my applications
GET	`/api/applications/employer/{id}`	Employer	Get applications for my jobs

PUT	`/api/applications/{id}/status`	Employer	Update application status
DELETE	`/api/applications/{id}`	Employer/Admin	Delete application

9.4 Messaging

Method	Endpoint	Auth	Description
GET	`/api/messages/inbox/{userId}`	User	Get inbox with unread counts
GET	`/api/messages/conversation/{userId}`	User	Get conversation with user
POST	`/api/messages/send`	User	Send a message
GET	`/api/messages/unread-count/{userId}`	User	Get unread message count

9.5 Resume

Method	Endpoint	Auth	Description
POST	`/api/resume/upload`	User	Upload resume PDF
GET	`/api/resume/user/{userId}`	User	Get user's resumes
POST	`/api/resume/analyze/{resumeId}`	User	ATS analysis of resume
POST	`/api/resume/match/{resumeId}/{jobId}`	User	Match resume with job
DELETE	`/api/resume/delete/{resumeId}`	User	Delete resume

9.6 Admin

Method	Endpoint	Auth	Description
GET	`/api/admin/users`	Admin	List all users
GET	`/api/admin/jobs`	Admin	List all jobs
GET	`/api/admin/stats`	Admin	Platform statistics
PUT	`/api/admin/users/{id}/role`	Admin	Change user role
DELETE	`/api/admin/users/{id}`	Admin	Delete user

DELETE	`/api/admin/jobs/{id}`	Admin	Delete job

10. Technology Stack

Backend

Technology	Version	Purpose
Java	25	Programming language
Spring Boot	4.0.4	Application framework
Spring Security	7.0.6	Authentication & authorization
Spring Data JPA	—	Data access abstraction
Hibernate	7.2.7	ORM framework
MySQL Connector/J	—	Database driver
WebSocket (STOMP)	—	Real-time communication
JavaMailSender	—	Email notifications
Lombok	—	Boilerplate reduction
Maven	—	Build automation

Frontend

Technology	Version	Purpose
React	19	UI component library
Vite	7	Build tool & dev server
Tailwind CSS	—	Utility-first CSS framework
Zustand	—	State management
Axios	—	HTTP client
Lucide React	—	Icon library
Recharts	—	Charting library

React Router	—	Client-side routing

11. Security Features

Password Encryption

BCrypt hashing via Spring Security PasswordEncoder — passwords never stored in plain text

JWT Authentication

Stateless token-based auth with 24-hour expiration, signed with HS256 algorithm

Role-Based Access

@PreAuthorize annotations enforce role checks on every protected endpoint

CORS Configuration

Whitelisted origins restrict API access to authorized frontend domains only

Input Validation

@Valid annotations and custom validators on all request bodies

SQL Injection Prevention

All queries use JPA parameterized queries — no raw SQL concatenation

XSS Prevention

React auto-escapes all output; no dangerouslySetInnerHTML used

File Upload Validation

PDF-only restriction, file type and size validation before upload

12. Future Enhancements

- → **OAuth2 Login** — Google, GitHub, LinkedIn single sign-on
- → **AI Resume Parsing** — LLM-based skill extraction and structured data parsing
- → **Video Interviews** — Integrated video calling for remote interviews
- → **Push Notifications** — Browser push and mobile push notifications
- → **Mobile App** — React Native companion app for iOS and Android
- → **Advanced Analytics** — Machine learning-based job-candidate matching
- → **Multi-language Support** — Internationalization (i18n) for global reach
- → **Payment Integration** — Premium job postings and featured listings
- → **Candidate Scoring** — AI-based candidate ranking for employers
- → **Interview Scheduling** — Calendar integration with Google/Outlook
- → **Application Status Timeline** — Visual timeline of every status change
- → **Employer Dashboard** — Aggregate analytics across all job postings